

uVRR Supplement S1: The polite2 Calm Maintenance Ladder, Measured

Evidence details, anchors, and provenance for the paper’s “Maintenance operations” section — full

ladder, second edition

Simon Massey

Abstract—This supplement carries the detail the paper abridges, in its second edition covering the complete maintenance ladder. The paper reports abdication failover at 26–30 ms per cycle, the timeout-driven takeover at 20,370 ms, the whole-cluster reboot serving at 2,596 ms, the live member swap at 23,542 ms, and the crash-restart scenario recorded blocked-with-evidence. Here: the exact run configuration, the anchor tables every number is computed from, the raw log lines and flight-tape records that witness each transition, the client-continuity computation, the blocked scenario with its diagnostics and its safety analysis, the session lineage (which agent session ran what, and how the figures in this document were recomputed from the raw artefacts rather than taken from session claims), and the reproduction path. Every claim carries a pointer to the lab book in `papers/results/` whose executable record evidences it; the lab books are the leaf layer of the paper → supplement → lab book → snapshot chain.

I. THE RUN AND ITS CONFIGURATION

The run is `local-polite2-2026-09-19`: a three-node `lunet-locks` cluster on one host (macOS, loopback), calm configuration so the baseline churn of the tight-timeout cells is absent — heartbeat 5 ms, election timeout 3,000 ms (sloppy detector 3,000–5,000 ms), recovery 3,000 ms. The binary is the flight-recorder build at commit `7ff3f12`; every node’s tape was growing before the scenarios began. Load throughout: one lease-load client per node, polite cadence, roughly 17 operations per second across the cluster (measured: 432/442/456 lease operations at `n1/n3/n2` over the 24.7 s `S2` window).

The ladder, in order: **S0** steady serving (240 s) → **S1** security-patch rotation through the non-leaders (SIGTERM/stop/restart, one node at a time) → **S2** leader abdication cycled three times so every node holds leadership once → **S3** leader SIGTERM/restart (the timeout-driven takeover baseline against `S2`) → **S4** SIGKILL crashes, leader last → **S5** full-cluster stop/start → **S6** controlled reconfiguration swap (explicit remove+add).

The campaign ran three times. Run A (snapshot 06:10:26Z) aborted at its steady gate: 26 client errors, traced to two host-side bugs in the run’s own custom stream client (a malformed GET line and a message-id generator with only 16 distinct rotations) plus one 18 s all-server timeout under investigation. Run B (snapshot 06:12:40Z) completed `S0`, `S1`, and `S2` — the numbers of Section IV — and stopped at `S3` for a harness

reason: the serving probe was single-shot where the gate needed retry-to-deadline, and `S4`’s rejoin budgets were below the measured era-fold walk (20–56 s). Run C, with both gates fixed, completed `S3` en route to `S4` at the time of writing (Section V). Each run’s full artefact set was snapshotted by `snapshot_run` (logs, superblock images, flight tapes, anchors, progress log) before the run directory was wiped; the three archives are the evidence base of record.

II. S0: STEADY STATE

240 s under load, anchor `steady-serving-begin` at 1789798363020 (run B): zero client errors; all three nodes `normal:v0:leader=n1` at the window’s end; no view change emitted. This is the calm baseline the tight-timeout softball cells never had: no suspicion ever fired.

III. S1: NON-LEADER ROTATION

Each target was SIGTERM’d (the node’s clean-stop path drains and flushes, then exits; the boot gate records the controlled halt) and restarted on the same boot line immediately. The measured rows:

TABLE I
S1: STOP/START ROTATION THROUGH NON-LEADERS (RUN B).

Node	stop (ms)	rejoin to normal (ms)	peer first-ok gap (ms)	rejoined at view
n2	50	14,988	84	41
n3	50	19,461	continued	91

The stop is the two marker-write rounds of the boot gate, 50 ms end-to-end. The rejoin tail is not the stop: the restarted node returns at the boot view and must fold the eras and views the live cluster produced in its absence — `n2` folded to view 41, `n3` to view 91 — and the current harness walks the fold at its own cadence. The stop/start number (50 ms) is the maintenance claim; the tail is the known cost of absence-time churn, and it is exactly what the paper’s `E1` design measures at sample counts.

IV. S2: ABDICATION, MEASURED END TO END

A. What is being measured

The paper’s claim under test: failover driven by abdication is detection-independent — bounded by the proposal round-trip,

Simon Massey (email: simon.massey@stenographer.cloud).

not by any timeout — because the abdication itself proposes the view change. The measured quantity per cycle is the span from the operator verb’s reply (S2-abdicate-drive, reply={"action": "abdicate", "accepted": true}) to the installation of the successor view’s first commit, plus the settle span to all three nodes reporting `normal` in the new view.

B. The anchor table (run B, verbatim)

TABLE II
S2 ANCHORS: THE ABDICATION CYCLES, MILLISECOND TIMESTAMPS.

Cycle	S2-begin	abdicate-drive	S2-settled
1 (leader n1)	1789802982055	1789802982066	1789802982200
2 (leader n2)	1789802994314	1789802994322	1789802994467
3 (leader n3)	1789803006601	1789803006608	1789803006757

The derived measures, computed from the anchors by the runner and verified by hand against the node logs:

TABLE III
S2 DERIVED MEASURES (MS).

Cycle	drive (begin→reply)	failover (reply→first commit)	settle (begin→settled)
1	8	30	145
2	8	27	153
3	7	26	156

The failover row is the paper’s number. The wire witness at the log line (cycle 1, node n2’s log): the abdication drive answered at `ts=1789802982066`; n2 logged `leader=2 era=1 view=85 ts=1789802982073` — the successor view’s designated primary inside 7 ms, the failure detector reading `phi 0.000` and never firing (lab book 02-abdication-failover.md, “the wire”; raw windows papers/results/data/s2-window-n*.txt). The rotation covered all three nodes: `n1 → n2 → n3 → n1`, views `84 → 85 → 86`.

C. Client continuity through the rotation

The window from cycle-1’s drive (1789802982066) to cycle-3’s settle (1789803006757) is 24,691 ms. Recomputed from the raw lease-attempt lines of the three node logs (not from session claims):

TABLE IV
CLIENT OPERATIONS IN THE S2 WINDOW, PER NODE.

Node	operations	mean interval (ms)	longest gap (ms)
n1	414	60	166
n2	440	56	170
n3	438	56	171

Total 1,292 operations at roughly 52/s cluster-wide (the run’s ~17 ops/s per node), zero client errors (recomputation: papers/results/data/s2-client-continuity.json). The longest gap at any node, 171 ms, is three mean intervals: the rotation is client-visible only as a momentary slowdown at worst, against the multi-second stall a timeout-driven takeover would impose.

D. The abdicating leader’s return

Each cycle’s settled states show the abdicating leader back normal in the successor view within the settle time (150–181 ms): it stops accepting new operations at the abdication, the successor view installs, and the old leader rejoins as an ordinary member. No node left the cluster; no announcement, no forced sequence, no weight change participated.

V. S3: THE TIMEOUT-DRIVEN BASELINE

The completed ladder’s S3 row (leader n1 SIGTERM’d; the runner’s measure line verbatim): stop 50 ms; the takeover — successor n2 elected and serving — at 20,370 ms; n1’s rejoin to `normal` at 42,991 ms. The takeover spans the sloppy detector’s window (3,000–5,000 ms) plus the era-fold walk the new leader’s catch-up required; the rejoin tail matches the S1 shape (churn accumulated during absence, walked at the harness cadence; lab book 02-abdication-failover.md, “the contrast”).

The contrast is the paper’s point: the same leadership change costs 26–30 ms when the leader abdicates and 20.4 s when the cluster must detect its death — three orders of magnitude apart on identical hardware and knobs.

The softball campaign’s attempt-3 row (tight timeouts, heart-beat 5 ms, election 1,000 ms, leader SIGTERM): successor elected correctly; the restarted leader’s rejoin took approximately 78 s against a 30 s gate — the boot-fenced chase of a churning view space, the failure class the paper’s E1 analyses.

VI. S4: THE CRASH-RESTART SCENARIO, BLOCKED WITH EVIDENCE

The ladder’s S4 SIGKILL’d all three nodes in turn (non-leaders first, the leader last), restarting each on the same boot line. The boot-gate classification worked on every crash: each reboot read its fence, found no clean-stop marker, and bumped its identity into the high band (old identities 1/2/3, new own=16777217/16777218/16777219, incarnation=1; witness lines papers/results/data/s4-bump-witness.txt).

Each bumped node’s first act was the reincarnation announcement to all three peers in the millisecond of its boot (its flight tape opens with the host’s reincarnate drive and the tag-0x0d frames; papers/results/data/s4-bumped-tape-head.txt).

The rejoin walk then wedged, on all three crashes. The leader’s log names the refusal: `ReincarnationRefused { sender: NodeId(2), ...}` — the announcement arriving attributable to the OLD identity where the bumped node announced from the NEW one; the harness’s transport identity mapping sits between the two, and the engine declines to reconfigure on a message it cannot attribute — the safe direction to fail, repeated 686 times across the three nodes’ logs (227/282/177 at n1/n3/n2; samples papers/results/data/s4-refusal-samples.txt). The forced walk never started (remap notices at zero peers), the bumped nodes sat fenced for the full 150 s budget, and the serving pair’s views stormed (1,431 and 1,146 leader-change records across the run on n1 and n3 — the era-fold

walk hunting a shape the refused announcement never gave it; lab book 03-crash-reincarnation.md, “where the rig stopped”).

The safety statement, checked on the tapes. The ladder closed with `total_client_errors=0`: no committed operation was lost anywhere in the run. No superseded identity voted again — the refusals are the protocol declining to act. The defect is a liveness defect of the harness’s rejoin walk, host-side by the repository’s bug-provenance ruling, localised to the transport identity mapping; the repair is the downstream repository’s active work. The $2 \times \text{RTT}$ crash-reincarnation floor the paper claims is the protocol’s, evidenced per piece (classification, bump, announcement, and the fused-reconfiguration machinery measured healthy in S6’s 28 ms decrement round), with this end-to-end block named.

VII. S5: THE WHOLE-CLUSTER REBOOT

Two passes, both recorded. Pass 1 attempted the durable resurrection of the post-S4 state — three nodes holding divergent retained views from the view storms (one at view 1,383, the others fenced at 0): the cluster did not converge in 120 s (all three `restarting v0`, 0/3 voters; the same era-fold family as Section VI). Pass 2 did what an operator does: snapshot for the record, re-provision fresh, reboot. All down 30 s; all up; all three voters `normal` at 2,578 ms and the first client operation served at 2,596 ms (anchors `papers/results/data/s5-anchors.txt`; lab book 04-reboot-and-reconfig.md).

VIII. S6: THE CONTROLLED RECONFIGURATION SWAP

n3 was retired and n4 joined, live, as four committed reconfiguration rounds: decrement 28 ms (one round-trip — the same machinery as the abdication), leave 5,022 ms, join 5,017 ms, increment 5,026 ms (the 5-second rounds are the harness’s drive cadence between rounds); the swap completed in 23,542 ms with n4 serving at voting weight 11,383 ms in, the cluster serving throughout (the measure line verbatim in lab book 04; anchors `papers/results/data/s6-anchors.txt`).

IX. LIMITATIONS

The run’s own REPORT discloses, and this supplement repeats: the per-operation stream client (softball7’s shape) was dropped from the run after a scratch tooling probe of that shape lost replies intermittently ($\sim 4\%$ per attempt) against this rig on this host, while the shipped Rust clients did not; the tape shows the wedged operations committing at the leader with the reply emitted — the loss is in the scratch client’s connection handling, not the cluster, and the runner replaced it with the shipped `lease-client` serving probes. The load clients’ 5 s op-timeout tail (exactly three samples per client, in the S4/S5 outage windows) sits in the cumulative p99/max; the clean-window medians carry the mid-distribution. And Section VI’s block stands as the ladder’s principal incomplete scenario.

TABLE V
THE EVIDENCE CHAIN.

Artefact	Location
Run spec	.tmp/delegation/item25.2
Snapshots (runs A, B, and the completed ladder)	iCloud 2026/uVRR/runs/polite
Lab books (the executable evidence)	papers/results/00-index.
Live run directory	.tmp/telemetry/local-polite
Paper of record	papers/paper-20260919-3a
This supplement	papers/supplement-s1-polite
Runner session	opencode ses_f4832dab4ffeZ

X. EVIDENCE PACKAGE AND PROVENANCE

The lineage, stated plainly. The run was executed by the runner session (“Polite-2 run with abdication”, 988-line roll-out): it built the abdication verb (core $\rightarrow$ adapter $\rightarrow$ FFI $\rightarrow$ Teal admin verb $\rightarrow$ rig and lease-client), the Teal ladder runner (`tools/lib/polite2.tl`), and the runbook, drove runs A–C and the completed ladder, and left each snapshot via `snapshot_run`. This supplement was then written against the snapshots themselves: every figure in Sections II–VIII was recomputed from the raw `anchors.md`, the node logs’ timestamped lines, and the flight tapes’ commit stamps — the session’s own claims were used only to locate the evidence, never as the evidence. The failure classes the runner fixed en route are recorded because they are part of the measurement’s story: the stream-client wedge was a host-side harness bug (16-rotation message-id generator, diagnosed through the flight tapes), not a protocol event; the S3/S4 gate fixes were harness budgets, not production changes. The lab books of `papers/results/` carry the executable layer of this chain: each one verifies under `showboat verify` against the committed `data/` extracts.

XI. REPRODUCTION

The ladder is a single command against the committed runner: `lua jit examples/lease-sequencer/run-polite2.lua`, which boots the calm configuration, drives the scenarios in order, writes the anchor and progress logs, and snapshots the run directory at the end. The knobs are verbatim in `anchors.md`’s header (heartbeat 5 ms, election 3,000 ms, recovery 3,000 ms). The figures of this supplement are recomputed from the snapshots by the method of Section X: anchors give the spans; the raw `lease-attempt` and `leader` log lines give the wire witnesses; the flight tapes give the commit stamps.